

ECOM EXPERTS

Hestiacp

Manual de usuario, tecnico y de desarrollo

hestiacp · 08/09/2026

Contenido

Tabla de contenidos

Características

Estructura del repositorio

Requisitos

Instalación rápida (Windows → servidor)

Instalación manual (en el servidor)

Kill switches (lee esto antes de usar)

Plugin Watchdog

Comandos CLI

Checks de monitorización

Configuración (conf/watchdog.conf)

Plugin Staging

Comandos CLI

Flujo de trabajo

Configuración (conf/staging.conf)

Interfaz web del panel

Actualización y desinstalación

Solución de problemas

Seguridad

Licencia

1. Requisitos previos

2. Estructura de instalación en el servidor

3. Subir los ficheros al servidor

4. Permisos

5. Ejecutar el instalador

6. Configuración inicial y KILL SWITCH (¡leer antes de usar!)

Watchdog (no enviará alertas hasta desactivar el kill switch)

Staging (no hará push-to-live hasta desactivar el kill switch)

7. Detalle del registro de comandos (referencia)

8. Integración con la interfaz web del panel (opcional)

9. Verificación post-instalación

10. Uso básico (smoke test)

11. Activar depuración (si algo falla)

12. Actualización de los plugins

13. Desinstalación

14. Solución de problemas frecuentes

15. Recordatorio de seguridad

0. Contexto y arquitectura de HestiaCP (lectura previa obligatoria)

Fase 1 — Alcance, casos de uso y modos de staging

Fase 2 — Diseño de arquitectura del plugin

Fase 3 — Modelo de configuración y plantillas

Fase 4 — Detección y lectura del entorno origen

Fase 5 — Creación del dominio/usuario de staging

Fase 6 — Clonado de ficheros

Fase 7 — Clonado de bases de datos

Fase 8 — Reescritura de configuración y URLs

Fase 9 — SSL, DNS y accesibilidad del staging

Fase 10 — Sincronización incremental (refresh de staging)

Fase 11 — Push-to-live: diseño seguro (CRÍTICO)

Fase 12 — Push-to-live: implementación y rollback

Fase 13 — Gestión del ciclo de vida de los entornos

Fase 14 — Comandos CLI del plugin (estilo HestiaCP)

Fase 15 — Interfaz web integrada en el panel

Fase 16 — Logging, depuración y auditoría

Fase 17 — Pruebas y validación de escenarios

Fase 18 — Instalación, compatibilidad y endurecimiento

Fase 19 — Documentación y entrega

Anexo A — Matriz "qué se copia" por modo

Anexo B — Riesgos clave y mitigaciones

0. Contexto y arquitectura de HestiaCP (lectura previa obligatoria)

Fase 1 — Definición de alcance, requisitos y catálogo de checks

Fase 2 — Diseño de la arquitectura del plugin

Fase 3 — Modelo de configuración

Fase 4 — Capa de utilidades y librería común (lib/)

Fase 5 — Implementación de checks de servicios y recursos

Fase 6 — Implementación de checks de servicios web y BBDD

Fase 7 — Checks de correo, DNS y certificados

Fase 8 — Checks de backups, seguridad e integridad

Fase 9 — Motor de alertas: anti-duplicado, ventanas y kill switch (CRÍTICO)

Fase 10 — Notificadores multicanal

Fase 11 — Orquestador y planificación (cron)

Fase 12 — Comandos CLI del plugin (estilo HestiaCP)

Fase 13 — Interfaz web integrada en el panel

Fase 14 — Sistema de logging, depuración y rotación

Fase 15 — Pruebas, simulación de fallos y validación

Fase 16 — Auto-recuperación (self-healing) opcional y segura

Fase 17 — Instalación, desinstalación y compatibilidad

Fase 18 — Documentación, hardening final y entrega

Anexo A — Matriz de checks (resumen)

Anexo B — Riesgos clave

Dos plugins autocontenidos para [HestiaCP](#) que añaden **monitorización del servidor** (Watchdog) y **clonado de dominios/BBDD con push-to-live** (Staging), integrados en la UI del panel y operables desde la línea de comandos con comandos nativos `v-*`.

HestiaCP 1.6+



Bash POSIX

Deploy

PowerShell + plink

License

AFL-3.0

Tabla de contenidos

- Características
- Estructura del repositorio
- Requisitos
- Instalación rápida (Windows → servidor)
- Instalación manual (en el servidor)
- Kill switches (lee esto antes de usar)
- Plugin Watchdog
- Plugin Staging
- Interfaz web del panel
- Actualización y desinstalación
- Solución de problemas
- Seguridad
- Licencia

Características

Watchdog (vigilancia del servidor) - 9 *checks* nativos: recursos, servicios, base de datos, HTTP/HTTPS, SSL, DNS, cola de Exim, backups y seguridad básica. - Umbrales configurables (CPU/RAM/disco/carga) con escalado WARNING → CRITICAL. - 5 canales de notificación: Email, Telegram, Slack, Discord y webhook genérico. - Anti-inundación: ventana anti-duplicado, *rate limit* por hora y *digest* por ejecución. - Cron cada 5 min registrado con la herramienta nativa `v-add-cron-job`.

Staging (clonado de dominios y BBDD) - Clona un dominio de producción a un entorno de staging (subdominio, dominio o usuario aparte). - Reescritura *serialize-safe* de URLs para WordPress, PrestaShop, Laravel, Joomla y estáticos. - Push-to-live **destructivo** protegido por kill switch + backup obligatorio + rollback. - Staging protegido por defecto: `noindex` + HTTP Basic Auth + pasarelas/emails desactivados.

Comunes - Idempotentes: re-ejecutar `install.sh` no duplica cron, comandos ni estado. - **Kill switches activados por defecto:** nada sale al exterior ni sobrescribe producción sin confirmación. - Integración en la UI del panel (menú superior + botones en *Server Settings*) con backup y validación `php -1`. - Sin dependencias de Composer; solo Bash y utilidades estándar del sistema.

Estructura del repositorio

```

hestiacp/
├─ deploy.ps1      # Despliegue desde Windows vía PuTTY/plink (stdin, sin SFTP)
├─ deploy.sh       # Despliegue desde Linux/macOS
├─ INSTALL.md      # Guía de instalación manual detallada
├─ PLAN_WATCHDOG.md # Diseño técnico del Watchdog
├─ PLAN_STAGING.md # Diseño técnico del Staging
├─ watchdog/
│   ├── install.sh # Instalador idempotente
│   ├── bin/v-watchdog-* # Comandos CLI
│   ├── checks/*.sh # Checks de monitorización
│   ├── notifiers/*.sh # Canales: email, telegram, slack, discord, webhook
│   ├── lib/        # common.sh, alerts.sh
│   ├── conf/watchdog.conf
│   └─ web/index.php # UI del panel
└─ staging/
    ├── install.sh # Instalador idempotente
    ├── bin/v-staging-* # Comandos CLI
    ├── lib/*.sh      # clone_db, clone_files, push_live, rollback, sync, rewrite, ssl_dns...
    ├── conf/staging.conf
    └─ web/index.php  # UI del panel
  
```

Tras la instalación, los plugins viven en `/usr/local/hestia/plugins/<plugin>/` y los comandos se publican en `/usr/local/hestia/bin/` (ya en el PATH del panel).

Requisitos

- HestiaCP **1.6+** sobre Debian 11/12 o Ubuntu 20.04/22.04/24.04.
- Acceso **root** por SSH.
- Paquetes (normalmente ya presentes): `bash`, `curl`, `flock`, `rsync`, cliente `mysql` / `mariadb` y/o `psql`.
- Opcional (staging WordPress): **WP-CLI** para `search-replace` `serialize-safe`.
- Para el despliegue desde Windows: **PuTTY** (`plink.exe`) y `tar.exe` (incluido en Windows 10/11).

Instalación rápida (Windows → servidor)

El script `deploy.ps1` empaqueta `watchdog/` y `staging/`, los sube por **stdin sobre plink** (no usa pscp/SFTP, que muchos servidores tienen deshabilitado), normaliza saltos de línea, ajusta permisos, ejecuta los instaladores y publica la UI. Es idempotente y **preserva** `conf/` y `state/`.

1. Instala PuTTY si no lo tienes: `powershell winget install --id PuTTY.PuTTY -e`

2. Edita la cabecera de `deploy.ps1` con los datos del servidor: `powershell $Server = '00.00.00.00' # IP o host del servidor HestiaCP $User = 'root' $Pass = '...' # contraseña root del servidor destino $Port = '22'`
3. Ejecútalo: `powershell .\deploy.ps1` # si la política lo bloquea: `powershell -ExecutionPolicy Bypass -File .\deploy.ps1`

El script realiza 6 pasos y termina mostrando los kill switches y las URLs de la UI:

```
[0/6] Cacheo de host key      [3/6] LF + sincronización + permisos
[1/6] Verificación HestiaCP  [4/6] Instaladores (cron + menú + botones)
[2/6] Empaquetado y subida   [5/6] Publicación de UI (symlinks)
                             [6/6] Verificación final
```

Desde Linux/macOS existe el equivalente `deploy.sh`.

Instalación manual (en el servidor)

Resumen; la guía completa está en [INSTALL.md](#).

```
# 1) Subir las carpetas (ajusta TU_SERVIDOR)
scp -r ./watchdog ./staging root@TU_SERVIDOR:/usr/local/hestia/plugins/

# 2) Normalizar saltos de línea (si se editaron en Windows)
apt-get install -y dos2unix
find /usr/local/hestia/plugins/{watchdog,staging} -type f \
  \( -name '*.sh' -o -name 'v-*' \) -exec dos2unix {} \;

# 3) Ejecutar los instaladores (idempotentes)
cd /usr/local/hestia/plugins/watchdog && bash install.sh
cd /usr/local/hestia/plugins/staging && bash install.sh
```

Kill switches (lee esto antes de usar)

Ambos plugins arrancan **bloqueados** por seguridad. No envían nada al exterior ni sobrescriben producción hasta que los desactivas conscientemente.

Plugin	Clave	Por defecto	Efecto mientras está <code>true</code>
Watchdog	<code>WD_KILL_SWITCH</code>	<code>'true'</code>	Los checks se ejecutan y registran, pero no se envía ninguna notificación.
Staging	<code>STG_PUSH_KILL_SWITCH</code>	<code>'true'</code>	Crear/sincronizar staging es seguro; el push-to-live está bloqueado.

```
# Validar canales del watchdog y activar el envío real
v-watchdog-test-notify telegram
v-watchdog-config-set WD_KILL_SWITCH 'false'

# Solo cuando vayas a publicar a producción, y tras tener backup verificado
v-staging-config-set STG_PUSH_KILL_SWITCH 'false'
```

Plugin Watchdog

Comandos CLI

Comando	Función
<code>v-watchdog-run</code>	Orquestador: ejecuta los checks y dispara las alertas (lo invoca el cron).
<code>v-watchdog-status</code>	Estado agregado (semáforo por categoría).
<code>v-watchdog-list-alerts</code>	Lista las alertas registradas en el journal.
<code>v-watchdog-ack</code>	Marca alertas como reconocidas (ACK).
<code>v-watchdog-test-notify</code>	Envía una notificación de prueba por los canales.
<code>v-watchdog-config-get</code>	Lee valores de <code>conf/watchdog.conf</code> .
<code>v-watchdog-config-set</code>	Escribe/actualiza una clave de configuración.
<code>v-watchdog-debug</code>	Activa/desactiva la depuración y muestra diagnóstico.

```
v-watchdog-run --dry-run      # pasada en seco (no notifica)
v-watchdog-status             # semáforo por categoría
v-watchdog-list-alerts        # historial de alertas
```

Checks de monitorización

Check	Qué vigila
resources	CPU, RAM, disco y carga del sistema (umbrales configurables).
services	Servicios del sistema (nginx, apache, mysql, exim...).
database	MariaDB/MySQL y PostgreSQL.
http	Disponibilidad HTTP/HTTPS de los dominios alojados.
ssl	Caducidad de certificados SSL por dominio.
dns	Resolución DNS de zonas gestionadas por bind/named.
exim_queue	Tamaño de la cola de correo de Exim.
backups	Antigüedad del último backup por usuario.
security	Comprobaciones de seguridad básicas.

Configuración (`conf/watchdog.conf`)

Umbrales (`WD_CPU_WARN/CRIT` , `WD_RAM_*` , `WD_DISK_*` , `WD_LOAD_*`), ventana anti-duplicado (`WD_NOTIFY_WINDOW`), *rate limit* (`WD_RATE_MAX_HOUR`), escalado (`WD_ESCALATE_AFTER`), *digest* (`WD_DIGEST`) y los canales:

```
WD_CHANNEL_TELEGRAM='true'
WD_TELEGRAM_TOKEN='123456:ABC...'
WD_TELEGRAM_CHAT_ID='987654321'
```

Canales soportados: **Email**, **Telegram**, **Slack**, **Discord**, **webhook** genérico. Los secretos viven en el `.conf` con permisos `640` y **nunca** se escriben en los logs.

Plugin Staging

Comandos CLI

Comando	Función
<code>v-staging-create</code>	Crea un entorno de staging a partir de un dominio de producción.
<code>v-staging-sync</code>	Re-sincroniza producción → staging (ficheros y/o BBDD).
<code>v-staging-list</code>	Lista los entornos de staging registrados.
<code>v-staging-info</code>	Detalle completo de un entorno.
<code>v-staging-push</code>	PUSH-TO-LIVE: promociona staging a producción (destructivo, protegido).
<code>v-staging-rollback</code>	Revierte un push-to-live dejando producción como estaba.
<code>v-staging-delete</code>	Elimina únicamente el entorno de staging (web + BBDD). Nunca toca el origen.
<code>v-staging-debug</code>	Diagnóstico: entorno, dependencias y estado.

Flujo de trabajo

```
# 1) Crear un staging de un dominio existente (modo subdominio)
v-staging-create admin midominio.com --mode subdomain
v-staging-list
v-staging-info staging.midominio.com

# 2) Re-sincronizar cuando producción avance
v-staging-sync staging.midominio.com

# 3) Publicar a producción (requiere backup + kill switch desactivado)
v-staging-config-set STG_PUSH_KILL_SWITCH 'false'
v-staging-push staging.midominio.com
# Si algo sale mal:
v-staging-rollback midominio.com
```

Configuración (`conf/staging.conf`)

- **Seguridad:** `STG_PUSH_KILL_SWITCH` , `STG_BACKUP_TTL` (backup válido máx. 24 h antes de autorizar push).
- **Nomenclatura:** `STG_NAME_PATTERN` (`staging.{domain}`), `STG_DEFAULT_MODE` , prefijos de BBDD `STG_DB_PREFIX` / `STG_DBUSER_PREFIX` .
- **Protección:** `STG_NOINDEX` , `STG_BASIC_AUTH` , `STG_DISABLE_EMAILS` , `STG_DISABLE_PAYMENTS` .
- **CMS:** plantillas de reescritura para WordPress / PrestaShop / Laravel / Joomla y exclusiones de rsync (`STG_RSYNC_EXCLUDES` , `STG_UPLOADS_EXCLUDE`).

Interfaz web del panel

Los instaladores publican los accesos y la integran en la UI de HestiaCP:

- Menú superior: **STAGING** (todos los usuarios) / **WATCHDOG** (solo `admin`).
- *Server Settings* → *Configure*: sección **Watchdog & Staging** con botones de acceso.
- URLs directas (sesión iniciada en el panel):
 - `https://TU_SERVIDOR:8083/pluginwatchdog/`
 - `https://TU_SERVIDOR:8083/pluginstaging/`

La integración se **re-aplica en cada instalación** (las actualizaciones del panel sobrescriben los ficheros core) y siempre con backup `*.ecombak` y validación `php -1`: si el resultado no valida, se conserva el original.

Actualización y desinstalación

```
# Actualizar: subir la nueva versión y re-ejecutar el instalador (idempotente)
cd /usr/local/hestia/plugins/watchdog && bash install.sh
# -> conf/*.conf y state/ NO se sobrescriben

# Desinstalar
cd /usr/local/hestia/plugins/watchdog && bash uninstall.sh
cd /usr/local/hestia/plugins/staging && bash uninstall.sh
rm -f /usr/local/hestia/web/pluginwatchdog /usr/local/hestia/web/pluginstaging
```

`uninstall.sh` elimina los comandos de `bin/`, quita el cron y (preguntando) borra `state/` y `logs/`. **No** toca dominios, BBDD ni datos de producción.

Solución de problemas

Síntoma	Causa probable	Solución
Argument list too long al subir	(corregido en <code>deploy.ps1</code>) payload incrustado en el comando	El despliegue ya usa stdin ; actualiza a la versión actual del script.
<code>v-watchdog-run: command not found</code>	symlink no creado / PATH no refrescado	Re-ejecutar <code>install.sh</code> ; <code>hash -r</code> .
<code>bad interpreter: ^M</code>	finales de línea CRLF	<code>dos2unix</code> sobre los <code>.sh</code> .
No llegan alertas	kill switch activo	<code>v-watchdog-config-set WD_KILL_SWITCH 'false'</code> .
Push-to-live "bloqueado"	kill switch de staging activo	Backup + <code>STG_PUSH_KILL_SWITCH 'false'</code> .
Staging WordPress con URLs rotas	reemplazo no serialize-safe	Instalar WP-CLI .
UI da 403 / redirige al login	sesión no iniciada o no eres <code>admin</code>	Inicia sesión como <code>admin</code> en <code>:8083</code> .

Depuración detallada (no expone tokens):

```
v-watchdog-debug on # o v-staging-debug on
tail -f /usr/local/hestia/plugins/watchdog/logs/watchdog.log
v-watchdog-debug off
```

Seguridad

- **Kill switches activados por defecto:** nada sale al exterior ni sobrescribe producción sin confirmación explícita.
- **Backup obligatorio antes de push-to-live:** se exige un backup verificado (TTL configurable) y confirmación del dominio.
- **Origen intocable:** las operaciones de staging (create/sync/delete) nunca modifican el dominio de producción de origen.
- **Secretos protegidos:** tokens y contraseñas en `conf/*.conf` con permisos `640`, nunca en los logs.
- **No subas credenciales reales:** `deploy.ps1` incluye `$Server` / `$Pass` de ejemplo. Rellénalos en tu copia local y mantenlos fuera del control de versiones.

Licencia

AFL-3.0 · © 2026 Ecom Experts · ecomyseo@gmail.com

Guía de Instalación — Plugins HestiaCP (Watchdog y Staging)

Instrucciones para instalar, integrar y desinstalar los plugins **watchdog** (vigilancia) y **staging** (duplicación de dominios/BBDD) en un servidor con HestiaCP. Todos los comandos se ejecutan como **root** (o con `sudo`) por SSH.

1. Requisitos previos

- HestiaCP instalado y funcionando (Debian 11/12 o Ubuntu 20.04/22.04/24.04).
- Acceso **root** por SSH al servidor.
- Paquetes base (normalmente ya presentes en HestiaCP):
 - `bash`, `curl`, `flock` (util-linux), `rsync`.
- Cliente de BBDD: `mysql` / `mariadb` y/o `psql` (PostgreSQL) según uses.
- Opcional para staging de WordPress: **WP-CLI** (`wp`) para el `search-replace` `serialize-safe`.
- Espacio en disco suficiente (staging duplica ficheros + BBDD del dominio).

Comprobación rápida:

```
# Versión de HestiaCP
cat /usr/local/hestia/conf/hestia.conf | grep VERSION
# Herramientas necesarias
for c in bash curl flock rsync mysql; do command -v "$c" >/dev/null && echo "OK $c" || echo "FALTA $c"; done
# WP-CLI (opcional, solo staging WordPress)
command -v wp >/dev/null && echo "OK wp-cli" || echo "wp-cli no instalado (opcional)"
```

2. Estructura de instalación en el servidor

Los plugins se instalan bajo el árbol de HestiaCP. Los **comandos CLI** se publican en `/usr/local/hestia/bin/` (que ya está en el PATH del panel) y el código del plugin queda autocontenido en su carpeta:

<code>/usr/local/hestia/plugins/watchdog/</code>	<code># núcleo, checks, notifiers, conf, state, logs, web</code>
<code>/usr/local/hestia/plugins/staging/</code>	<code># núcleo, lib, conf, state, logs, web</code>
<code>/usr/local/hestia/bin/v-watchdog-*</code>	<code># comandos CLI (symlink o copia)</code>
<code>/usr/local/hestia/bin/v-staging-*</code>	<code># comandos CLI (symlink o copia)</code>

El directorio `/usr/local/hestia/plugins/` no es estándar de HestiaCP; lo crea el instalador. No interfiere con las actualizaciones del panel porque el upgrade de HestiaCP solo toca `bin/`, `web/`, `func/` e `install/`. Por eso los comandos `bin/v-*` se instalan como **symlink** al plugin (ver §7).

3. Subir los ficheros al servidor

Desde tu equipo (sustituye `TU_SERVIDOR`):

```
# Watchdog
scp -r ./watchdog root@TU_SERVIDOR:/usr/local/hestia/plugins/watchdog
# Staging
scp -r ./staging root@TU_SERVIDOR:/usr/local/hestia/plugins/staging
```

Importante (saltos de línea): los `.sh` deben tener finales de línea **LF** (Unix). Si los editaste en Windows, normalízalos en el servidor antes de instalar:

```
bash apt-get install -y dos2unix 2>/dev/null find /usr/local/hestia/plugins/watchdog
/usr/local/hestia/plugins/staging \ -type f \( -name '*.sh' -o -name 'v-*' \) -exec dos2unix {}
\;
```

4. Permisos

```
# Propietario root y permisos seguros
chown -R root:root /usr/local/hestia/plugins/watchdog /usr/local/hestia/plugins/staging
# Scripts ejecutables
find /usr/local/hestia/plugins/watchdog/bin /usr/local/hestia/plugins/staging/bin -type f -exec
chmod 750 {} \;
chmod 750 /usr/local/hestia/plugins/watchdog/*.sh /usr/local/hestia/plugins/staging/*.sh
# Config y estado: solo root (pueden contener tokens)
chmod 640 /usr/local/hestia/plugins/watchdog/conf/*.conf
/usr/local/hestia/plugins/staging/conf/*.conf
chmod 750 /usr/local/hestia/plugins/watchdog/state /usr/local/hestia/plugins/staging/state
```

5. Ejecutar el instalador

Cada plugin trae su `install.sh` (idempotente: puedes re-ejecutarlo sin duplicar cron ni estado). El instalador valida la versión de HestiaCP, crea `state/` y `logs/`, publica los comandos en `bin/` y registra el cron.

```
# Watchdog
cd /usr/local/hestia/plugins/watchdog && bash install.sh

# Staging
cd /usr/local/hestia/plugins/staging && bash install.sh
```

6. Configuración inicial y KILL SWITCH (¡leer antes de usar!)

Ambos plugins arrancan con su **kill switch activado** por seguridad. No harán ninguna acción hacia terceros ni escritura destructiva hasta que lo desactives conscientemente.

Watchdog (no enviará alertas hasta desactivar el kill switch)

```
# Revisar configuración
v-watchdog-config-get
# Configurar un canal (ejemplo Telegram) y umbrales
v-watchdog-config-set WD_TELEGRAM_TOKEN '123456:ABC...'
v-watchdog-config-set WD_TELEGRAM_CHATID '987654321'
# Probar el canal SIN depender del cron
v-watchdog-test-notify telegram
# Cuando todo esté validado, activar el envío real:
v-watchdog-config-set WD_KILL_SWITCH 'false'
# Ejecución de prueba sin notificar:
v-watchdog-run --dry-run
```

Staging (no hará push-to-live hasta desactivar el kill switch)

```
# El push a producción está bloqueado por defecto:
v-staging-config-get # STG_PUSH_KILL_SWITCH='true'
# Crear/sincronizar staging es seguro y NO requiere desactivarlo.
# Solo cuando vayas a publicar a producción, y tras tener backup:
v-staging-config-set STG_PUSH_KILL_SWITCH 'false'
```

7. Detalle del registro de comandos (referencia)

El `install.sh` publica los comandos como **symlink** para que sobrevivan a las actualizaciones de HestiaCP y se editen en un único sitio:

```
for f in /usr/local/hestia/plugins/watchdog/bin/v-*; do
    ln -sf "$f" "/usr/local/hestia/bin/$(basename "$f")"
done
for f in /usr/local/hestia/plugins/staging/bin/v-*; do
    ln -sf "$f" "/usr/local/hestia/bin/$(basename "$f")"
done
hash -r    # refrescar la caché de comandos del shell
```

El cron del watchdog se registra con la herramienta nativa del panel:

```
# Ejemplo: ejecución cada 5 minutos (lo hace install.sh automáticamente)
v-add-cron-job admin '*/5' '*' '*' '*' '*' '*' '/usr/local/hestia/bin/v-watchdog-run'
```

8. Integración con la interfaz web del panel (opcional)

Los plugins incluyen una UI en `plugins/<plugin>/web/index.php`. Para abrirla de forma sencilla y compatible con actualizaciones, publica un acceso bajo el árbol web del panel:

```
ln -sf /usr/local/hestia/plugins/watchdog/web /usr/local/hestia/web/pluginwatchdog
ln -sf /usr/local/hestia/plugins/staging/web /usr/local/hestia/web/pluginstaging
```

Accede (solo usuario `admin`, sesión iniciada en el panel):

- Watchdog: `https://TU_SERVIDOR:8083/pluginwatchdog/`
- Staging: `https://TU_SERVIDOR:8083/pluginstaging/`

La UI valida la sesión de HestiaCP y restringe el acceso a `admin`. Si tras un upgrade del panel desaparece el symlink, vuelve a crearlo (no se pierde nada).

9. Verificación post-instalación

```
# Comandos disponibles
command -v v-watchdog-run v-staging-create
# Estado del watchdog
v-watchdog-status
# Listado de entornos staging (vacío al inicio)
v-staging-list
# Logs
tail -n 50 /usr/local/hestia/plugins/watchdog/logs/watchdog.log
tail -n 50 /usr/local/hestia/plugins/staging/logs/staging.log
# Cron registrado
crontab -u admin -l | grep v-watchdog-run
```

10. Uso básico (smoke test)

```
# Watchdog: una pasada en seco (no notifica)
v-watchdog-run --dry-run

# Staging: crear un entorno de un dominio existente (modo subdominio)
v-staging-create admin midominio.com --mode subdomain
v-staging-list
v-staging-info staging.midominio.com
```

11. Activar depuración (si algo falla)

```
v-watchdog-debug on      # traza detallada en logs/ (sin exponer tokens)
v-staging-debug on
# ...reproducir el problema...
tail -f /usr/local/hestia/plugins/watchdog/logs/watchdog.log
v-watchdog-debug off
```

12. Actualización de los plugins

```
# Subir la nueva versión sobre la carpeta del plugin y re-ejecutar el instalador
scp -r ./watchdog root@TU_SERVIDOR:/usr/local/hestia/plugins/
cd /usr/local/hestia/plugins/watchdog && bash install.sh # idempotente
```

La configuración (`conf/*.conf`) y el estado (`state/`) **no se sobrescriben**: el instalador solo crea valores por defecto si no existen.

13. Desinstalación

```
# Watchdog
cd /usr/local/hestia/plugins/watchdog && bash uninstall.sh
# Staging
cd /usr/local/hestia/plugins/staging && bash uninstall.sh

# Limpiar symlinks de la UI (si los creaste)
rm -f /usr/local/hestia/web/pluginwatchdog /usr/local/hestia/web/pluginstaging
```

`uninstall.sh` elimina los comandos de `bin/`, quita el cron y (preguntando) borra `state/` y `logs/`. No toca dominios, BBDD ni datos de producción.

14. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
<code>v-watchdog-run: command not found</code>	symlink no creado / PATH no refrescado	Re-ejecutar <code>install.sh</code> ; <code>hash -r</code>
<code>bad interpreter: ^M</code>	finales de línea CRLF	<code>dos2unix</code> sobre los <code>.sh</code> (§3)
No llegan alertas	kill switch activo	<code>v-watchdog-config-set WD_KILL_SWITCH 'false'</code>
Push-to-live "bloqueado"	kill switch de staging activo	Hacer backup + <code>STG_PUSH_KILL_SWITCH 'false'</code>
<code>Permission denied</code> al ejecutar	script sin <code>+x</code>	<code>chmod 750</code> (§4)
Staging WordPress con URLs rotas	reemplazo no serialize-safe	Instalar WP-CLI ; el plugin usa <code>wp search-replace</code>
UI da 403 / redirige al login	sesión de panel no iniciada o no eres <code>admin</code>	Inicia sesión como <code>admin</code> en <code>:8083</code>

15. Recordatorio de seguridad

- **Kill switches activados por defecto:** nada sale al exterior ni sobrescribe producción sin tu confirmación explícita.

- **Backups antes de push-to-live:** el plugin exige un backup verificado de producción y confirmación del dominio antes de publicar.
- **Secretos:** tokens y contraseñas viven en `conf/*.conf` con permisos `640` y nunca se escriben en los logs.
- **Origen intocable:** las operaciones de staging (create/sync/delete) nunca modifican el dominio de producción de origen.

Plan de Desarrollo — Plugin Staging / Duplicación para HestiaCP

Documento de planificación técnica. Plugin que crea entornos de staging (clonado de un dominio web + sus bases de datos + ficheros + configuración) dentro de HestiaCP, con sincronización controlada y push-to-live seguro.

0. Contexto y arquitectura de HestiaCP (lectura previa obligatoria)

Para clonar un sitio hay que entender qué compone un "sitio" en HestiaCP:

- **Dominio web:** definido en `/usr/local/hestia/data/users/<user>/web.conf` ; raíz en `/home/<user>/web/<domain>/public_html/` ; configs de nginx/apache en `/home/<user>/conf/web/<domain>/` .
- **Bases de datos:** en `db.conf` del usuario; MariaDB/MySQL y/o PostgreSQL.
- **DNS:** zonas en `dns.conf` (bind) si el dominio usa DNS local.
- **SSL:** certificados Let's Encrypt o manuales asociados al dominio.
- **PHP:** versión por dominio (multi-PHP / php-fpm pools).
- **Comandos clave:** `v-add-web-domain` , `v-add-database` , `v-add-web-domain-alias` , `v-add-letsencrypt-domain` , `v-copy-*` (si existen), `v-dump-database` / `v-import-database` , `v-backup-user` , `v-restore-user` , `v-change-web-domain-php` , `v-list-*` .
- **Cuotas y planes:** los paquetes (packages) limitan número de dominios/BBDD; el staging debe respetarlos.

Estrategia base: un entorno de staging es **un nuevo dominio** (subdominio tipo `staging.midominio.com` o dominio aparte) con **copia de ficheros**, **copia de la(s) base(s) de datos** (con prefijo/usuario propios), **config PHP replicada** y, opcionalmente, **SSL y entrada DNS** propias. El push-to-live es la operación inversa, controlada y reversible.

Regla de oro de este proyecto: ninguna operación que toque datos de producción (sobre todo el *push-to-live* o el *drop* de BBDD) se ejecuta sin backup previo, confirmación explícita y posibilidad de rollback. Ver Fases 9-12.

Fase 1 — Alcance, casos de uso y modos de staging

- Casos de uso: clonar para pruebas, preparar una migración/upgrade de CMS, entorno de QA por rama, refresco periódico de staging desde producción.
- Modos soportados: 1. **Subdominio de staging** en el mismo usuario (más simple). 2. **Dominio independiente** (p. ej. `stg-cliente.tudominio.com`). 3. **Usuario/cuenta separada** (aislamiento total).

- Definir qué se clona: ficheros, BBDD, SSL, DNS, cron del dominio, pools PHP.
- Definir política de datos sensibles (anonimización opcional en staging).
- Entregable: documento de alcance con los modos y matriz de "qué se copia".

Fase 2 — Diseño de arquitectura del plugin

- Estructura propuesta: `/usr/local/hestia/plugins/staging/` |— `bin/` # `v-staging-*` (`create`, `sync`, `push`, `list`, `delete...`) |— `lib/` # funciones comunes (`clonado web`, `db`, `dns`, `ssl`) |— `conf/` # `staging.conf` (config global + plantillas) |— `state/` # registro de entornos staging y su origen |— `hooks/` # ganchos pre/post de cada operación |— `logs/` # logs del plugin |— `web/` # UI PHP integrada |— `install.sh` / `uninstall.sh`
- Componentes: **clonador de ficheros**, **clonador de BBDD**, **reescritor de config** (URLs, rutas, credenciales), **gestor de relaciones origen↔staging**, **motor de push-to-live**, **UI**.
- Contrato de metadatos: cada entorno staging guarda su `source_domain`, `source_user`, BBDD origen↔destino, fecha de creación y último sync.
- Entregable: diagrama de componentes y modelo de metadatos.

Fase 3 — Modelo de configuración y plantillas

- `staging.conf`: patrón de nombres (`staging.{domain}`, `{domain}-stg`), prefijos de BBDD/usuario, TTL del entorno, protección por contraseña (HTTP Basic), robots noindex por defecto, kill switch para push-to-live.
- Plantillas de transformación: lista de búsquedas/reemplazos por CMS (WordPress, PrestaShop, Joomla, Laravel `.env`, etc.).
- Mapa de exclusiones de ficheros (cachés, logs, `node_modules`, uploads grandes opcional).
- Entregable: `staging.conf` documentado + plantillas por CMS.

Fase 4 — Detección y lectura del entorno origen

- Dado un dominio, recolectar: usuario propietario, ruta `public_html`, versión PHP, BBDD asociadas (parsear `wp-config.php`, `.env`, `config/settings.inc.php` o consultar a partir de `db.conf`), SSL activo, registros DNS.
- Detección de CMS para aplicar la plantilla de reemplazos correcta.
- Cálculo de tamaño (ficheros + BBDD) y verificación de cuota disponible.
- Validaciones previas: dominio existe, usuario válido, espacio suficiente.
- Entregable: `lib/detect_source.sh` que devuelve un manifiesto del origen.

Fase 5 — Creación del dominio/usuario de staging

- Según modo elegido: `v-add-web-domain` (subdominio o dominio) o `v-add-user` + dominio para aislamiento.

- Replicar versión PHP con `v-change-web-domain-php` .
- Aplicar protección: HTTP Basic Auth y cabecera `X-Robots-Tag: noindex` .
- Crear estructura de directorios y permisos correctos del nuevo dominio.
- Registrar la relación origen↔staging en `state/` .
- Entregable: dominio de staging creado y accesible (vacío aún).

Fase 6 — Clonado de ficheros

- Copia con `rsync -a --delete` (consistente y reanudable) desde `public_html` origen al de staging, respetando exclusiones de Fase 3.
- Preservar permisos/propietario adaptados al usuario destino (chown).
- Soporte de copia incremental para `sync` posteriores (solo diferencias).
- Barra de progreso/registro de bytes transferidos y verificación final.
- **No** copiar mientras hay escritura intensa sin avisar; opción de modo mantenimiento del origen durante el primer clonado.
- Entregable: clonador de ficheros con modo full e incremental.

Fase 7 — Clonado de bases de datos

- Para cada BBDD del origen: `v-dump-database` (o `mysqldump` / `pg_dump`) → crear BBDD+usuario de staging con `v-add-database` → importar con `v-import-database` .
- Nombres/usuarios de staging con prefijo propio para evitar colisiones y respetar el límite de longitud de HestiaCP.
- Manejo de BBDD grandes: dump por tablas, compresión, importación por lotes.
- Verificación de integridad post-import (conteo de tablas/filas de control).
- **Regla anti-acción-masiva:** los dumps/imports operan solo sobre las BBDD del dominio indicado, nunca un barrido de todas las BBDD del servidor.
- Entregable: clonador de BBDD con verificación de integridad.

Fase 8 — Reescritura de configuración y URLs

- Aplicar plantilla de reemplazos: dominio origen → dominio staging en BBDD (con manejo de **serialización** de PHP en WordPress: usar WP-CLI `search-replace` o un reemplazo `serialize-safe`, nunca un `sed` plano sobre el dump).
- Actualizar credenciales de BBDD en `wp-config.php` / `.env` / `settings.inc.php` del staging.
- Ajustar rutas absolutas si las hubiera; regenerar caches/symlinks del CMS.
- Desactivar en staging: envío de emails reales, pasarelas de pago en modo producción, tareas cron destructivas, indexación.
- Entregable: staging funcional con URLs/credenciales correctas y "seguro".

Fase 9 — SSL, DNS y accesibilidad del staging

- Emitir SSL del subdominio staging con `v-add-letsencrypt-domain` (o cert autofirmado si es interno).
- Crear/ajustar registro DNS del subdominio si HestiaCP gestiona la zona; si no, documentar el registro a crear o usar `/etc/hosts` para pruebas internas.
- Validar que el staging responde correctamente (HTTP 200, login del CMS).
- Aplicar `noindex` y, opcionalmente, restricción por IP.
- Entregable: staging accesible vía HTTPS, no indexable y protegido.

Fase 10 — Sincronización incremental (refresh de staging)

- `v-staging-sync <domain>`: re-sincroniza ficheros (rsync incremental) y/o BBDD desde producción hacia el staging existente.
- Opciones selectivas: `--files-only`, `--db-only`, `--exclude-uploads`.
- **Backup del staging antes de sobrescribir** (por si la copia previa tenía trabajo manual a conservar).
- Re-aplicar reescritura de URLs/credenciales tras cada sync.
- Registro de fecha de último sync en el estado.
- Entregable: comando de sync incremental idempotente.

Fase 11 — Push-to-live: diseño seguro (CRÍTICO)

- **Kill switch hardcoded** `STG_PUSH_KILL_SWITCH=true` por defecto: bloquea cualquier push-to-live hasta que el usuario lo desactive conscientemente.
- **Backup obligatorio de producción** (`v-backup-user` o `dump+rsync` del dominio live) ANTES de cualquier escritura sobre live. Sin backup → no push.
- **Confirmación explícita** con doble verificación (escribir el nombre del dominio) y resumen de lo que se va a sobrescribir.
- Estrategia de mínima ventana: push de ficheros a un directorio temporal + swap atómico (symlink/rename) para evitar estado inconsistente.
- BBDD: import a una BBDD nueva y swap de credenciales, o import directo con backup previo verificado.
- Entregable: diseño documentado del push con todos los seguros.

Fase 12 — Push-to-live: implementación y rollback

- `v-staging-push <domain>`: ejecuta el push según el diseño de Fase 11.
- Reescritura inversa de URLs (staging → producción) `serialize-safe`.
- **Rollback automático**: si una etapa falla, restaurar desde el backup previo y dejar producción exactamente como estaba; notificar el resultado.
- `v-staging-rollback <domain>`: restauración manual del último backup live.
- Registro de auditoría completo de cada push (quién, cuándo, qué, resultado).

- Entregable: push-to-live con rollback probado en escenarios de fallo.

Fase 13 — Gestión del ciclo de vida de los entornos

- `v-staging-list` : listar entornos, su origen, tamaño, fecha y último sync.
- `v-staging-info <domain>` : detalle de un entorno (BBDD, rutas, estado).
- `v-staging-delete <domain>` : eliminar dominio + BBDD + usuarios del staging (con confirmación; nunca tocar el origen).
- **TTL / caducidad**: aviso y borrado opcional de stagings antiguos para liberar recursos (respetando cuotas del plan).
- Protección contra borrar accidentalmente un dominio de producción (verificar contra el registro de relaciones).
- Entregable: comandos de ciclo de vida con salvaguardas.

Fase 14 — Comandos CLI del plugin (estilo HestiaCP)

- `v-staging-create <user> <source_domain> [--mode subdomain|domain|user]`
- `v-staging-sync <domain> [--files-only|--db-only]`
- `v-staging-push <domain>` / `v-staging-rollback <domain>`
- `v-staging-list` / `v-staging-info` / `v-staging-delete`
- Salida `json|shell|plain`, códigos de retorno coherentes y `--help`.
- Bloqueo de concurrencia (flock) por dominio para evitar operaciones solapadas.
- Entregable: set completo de comandos CLI documentado.

Fase 15 — Interfaz web integrada en el panel

- Página en `web/` : listar dominios del usuario con botón "Crear staging".
- Asistente de creación (modo, qué clonar, anonimización, protección).
- Vista de cada staging: estado, último sync, botones Sync / Push / Eliminar.
- Push-to-live tras la UI con modal de confirmación fuerte + checkbox de "he hecho/autorizo backup" y mostrar que el kill switch debe estar off.
- Tokens CSRF, escape de salida en plantillas, permisos por rol/usuario.
- Entregable: UI completa e integrada con la navegación de HestiaCP.

Fase 16 — Logging, depuración y auditoría

- Logs en `plugins/staging/logs/` con nivel y rotación.
- Modo debug activable: traza de rsync, dumps, reemplazos y swaps (sin exponer contraseñas en claro).
- Log de auditoría separado para operaciones sensibles (push, delete).

- `v-staging-debug on|off` .
- Entregable: logging y auditoría robustos.

Fase 17 — Pruebas y validación de escenarios

- Pruebas con CMS reales: WordPress (serialización), PrestaShop, sitio estático, app Laravel con `.env` .
- Escenarios: crear → editar staging → sync (sin perder cambios indeseados) → push → verificar producción → rollback.
- Pruebas de fallo inducido en push (cortar a mitad) y verificar rollback total.
- Pruebas de cuota agotada, BBDD enorme, permisos, multi-PHP.
- Verificar que **nunca** se modifica el origen durante create/sync/delete.
- Entregable: suite de pruebas y reporte de escenarios cubiertos.

Fase 18 — Instalación, compatibilidad y endurecimiento

- `install.sh` con chequeo de versión de HestiaCP, dependencias (rsync, wp-cli opcional, clientes mysql/pg) y registro de comandos; `uninstall.sh` limpio.
- Compatibilidad nginx/apache, MariaDB/MySQL/PostgreSQL, Debian/Ubuntu soportados.
- Hardening: permisos de ficheros, validación/sanitización de nombres de dominio y BBDD, prevención de inyección en comandos, secretos fuera del repo.
- Idempotencia del instalador y de los comandos.
- Entregable: instalador probado en VM limpia + hardening verificado.

Fase 19 — Documentación y entrega

- Generar **3 documentos** `.md` : Técnico, Profesional de desarrollo y Guía de usuario (crear staging, sync, push-to-live, rollback, kill switch, FAQ).
 - Checklist final de seguridad: kill switch de push documentado, backup obligatorio verificado, rollback probado, origen intocable garantizado.
 - Guía de troubleshooting (serialización WP, permisos, SSL del subdominio, cuotas).
 - Entregable: paquete listo para producción + documentación completa.
-

Anexo A — Matriz "qué se copia" por modo

Elemento	Subdominio	Dominio aparte	Usuario aparte
Ficheros web	Sí	Sí	Sí
BBDD	Sí (prefijo)	Sí (prefijo)	Sí (aislada)
Versión PHP	Sí	Sí	Sí
SSL propio	Sí	Sí	Sí
DNS	Si zona local	Si zona local	Si zona local
Aislamiento	Bajo	Medio	Alto

Anexo B — Riesgos clave y mitigaciones

- **Sobrescribir producción por error** → kill switch + backup + doble confirmación + swap atómico + rollback (Fases 11-12).
- **Romper serialización de WordPress** → search-replace serialize-safe, nunca `sed` sobre el dump (Fase 8).
- **Staging indexado por Google** → noindex + Basic Auth por defecto (Fase 9).
- **Envío de emails/pagos reales desde staging** → desactivación forzada (Fase 8).
- **Barrido masivo de todas las BBDD** → operar solo sobre las del dominio (Fase 7).
- **Agotar disco/cuota** → cálculo previo y exclusiones (Fases 4 y 6).

Plan de Desarrollo — Plugin Watchdog para HestiaCP

Documento de planificación técnica. Sistema de vigilancia (watchdog) para HestiaCP que monitoriza servicios, recursos, dominios, certificados, colas de correo, backups y seguridad, con auto-recuperación y alertas multicanal.

0. Contexto y arquitectura de HestiaCP (lectura previa obligatoria)

HestiaCP es un panel de control de hosting (fork de VestaCP) basado en scripts Bash. Antes de escribir una línea hay que entender dónde encaja el plugin:

- **Binarios CLI:** `/usr/local/hestia/bin/v-*` — cada acción del panel es un comando (`v-list-sys-services` , `v-restart-service` , `v-list-users` , `v-add-cron-job` , etc.). El watchdog se apoya en estos, **no** reimplementa.
- **Configuración global:** `/usr/local/hestia/conf/hestia.conf` .
- **Datos de usuarios:** `/usr/local/hestia/data/users/<user>/` (web.conf, dns.conf, mail.conf, user.conf, stats, ...).
- **Hooks:** `/usr/local/hestia/data/hooks/` — scripts ejecutados ante eventos.
- **UI web:** PHP en `/usr/local/hestia/web/` (plantillas en `/usr/local/hestia/web/templates/`).
- **Cron del sistema:** HestiaCP gestiona tareas vía `v-add-cron-job` y el crontab del usuario `admin` .
- **Servicios típicos:** nginx, apache2, php-fpm (varias versiones), mariadb / mysql, postgresql, named/bind9, exim4, dovecot, spamd/clamav, fail2ban, hestia (panel).
- **Stack del panel:** Bash + PHP, sin frameworks pesados; preferir herramientas nativas del sistema (systemctl, ss, df, free, curl).

Regla de oro de este proyecto (anti-catástrofe): el watchdog hace acciones hacia terceros (envío de emails/Telegram/WhatsApp de alerta). Toda alerta debe estar protegida contra avalanchas: ventana máxima de re-notificación, control de estado anti-duplicado y kill switch. Ver Fase 9.

Fase 1 — Definición de alcance, requisitos y catálogo de checks

- Entrevista de requisitos: ¿qué se vigila y con qué prioridad?
- Catálogo inicial de comprobaciones (checks): 1. Servicios del sistema arriba/abajo. 2. Uso de CPU, RAM, swap y carga (load average). 3. Uso de disco por partición e inodos. 4. Estado de bases de datos (conexiones, réplica, tamaño). 5. Cola de correo de Exim (mensajes encolados, frozen). 6. Caducidad de certificados SSL (Let's Encrypt y manuales). 7. Caducidad de dominios (whois/expiry) — opcional. 8. Disponibilidad HTTP/HTTPS de los dominios alojados (código + latencia). 9. Resolución DNS de zonas gestionadas. 10. Estado y antigüedad de los backups de HestiaCP. 11. Procesos zombie / fugas de memoria de PHP-FPM. 12. Seguridad: intentos fail2ban, logins SSH, cambios en ficheros clave.

- Definir umbrales por defecto y severidades (INFO / WARNING / CRITICAL).
- Entregable: documento de requisitos + tabla de checks con umbrales.

Fase 2 — Diseño de la arquitectura del plugin

- Patrón: **núcleo de checks** + **motor de reglas** + **notificadores** + **persistencia de estado** + **UI**.
- Estructura de directorios propuesta: `/usr/local/hestia/plugins/watchdog/` — `bin/` # comandos `v-watchdog-*` — `lib/` # funciones bash compartidas (checks, utils) — `checks/` # un script por tipo de check — `notifiers/` # email, telegram, webhook, slack... — `conf/` # `watchdog.conf` (config global) — `state/` # estado anti-duplicado y last-run — `logs/` # logs propios del plugin — `web/` # UI PHP integrada en el panel — `install.sh` / `uninstall.sh`
- Decidir lenguaje de cada capa: checks y orquestación en **Bash**; UI en **PHP** (coherente con HestiaCP).
- Definir formato de salida interno de un check (JSON line o `clave=valor`) para que el motor de reglas lo consuma uniformemente.
- Entregable: diagrama de componentes y contrato de datos entre capas.

Fase 3 — Modelo de configuración

- Fichero `watchdog.conf` (formato `KEY='VALUE'` como HestiaCP) con: umbrales globales, intervalos, canales activos, kill switch, modo debug.
- Configuración granular por check (activar/desactivar, umbral propio).
- Configuración por host/dominio (excluir dominios de la monitorización HTTP).
- Validación de configuración al cargar (valores numéricos, rangos, canales).
- Soporte de overrides por entorno (variables al ejecutar el comando).
- Entregable: `watchdog.conf` de ejemplo documentado + parser/validador.

Fase 4 — Capa de utilidades y librería común (lib/)

- `wd_log()` — logging con nivel, timestamp y rotación.
- `wd_conf_get()` / `wd_conf_set()` — leer/escribir config de forma segura.
- `wd_state_get()` / `wd_state_set()` — persistencia de estado (último valor, última alerta, contador de fallos consecutivos).
- `wd_severity()` — normalización de severidades.
- Helpers para llamar a comandos `v-*` de HestiaCP capturando salida/errores.
- Manejo de errores y `set -euo pipefail` con traps.
- Entregable: `lib/common.sh` con tests unitarios (bats).

Fase 5 — Implementación de checks de servicios y recursos

- `checks/services.sh` : usa `v-list-sys-services` / `systemctl is-active` ; detecta servicios caídos y los que deberían estar activos según `hestia.conf`.
- `checks/resources.sh` : CPU (`/proc/stat`), RAM/swap (`free`), load (`uptime`), disco (`df`), inodos (`df -i`).
- Cada check devuelve estado normalizado + métrica + severidad.
- Umbrales configurables y "histéresis" para evitar flapping (X lecturas consecutivas antes de alertar).
- Entregable: checks de servicios y recursos funcionando vía CLI.

Fase 6 — Implementación de checks de servicios web y BBDD

- `checks/http.sh` : recorre dominios de `v-list-web-domains` de cada usuario, hace `curl` con timeout, valida código HTTP esperado, mide latencia y compara cadena de control opcional (keyword en el body).
- `checks/database.sh` : conexión a MariaDB/MySQL y PostgreSQL, número de conexiones, consultas lentas, tamaño, estado de réplica si aplica.
- `checks/phpfpm.sh` : pools activos, procesos, reinicios, memoria.
- **Regla anti-scan masivo:** la comprobación HTTP recorre dominios *actuales*, nunca histórico de pedidos/usuarios; y no dispara acción externa por dominio salvo a través del motor de alertas con anti-duplicado (Fase 9).
- Entregable: checks HTTP/DB/PHP-FPM operativos.

Fase 7 — Checks de correo, DNS y certificados

- `checks/exim_queue.sh` : tamaño de cola (`exim -bpc`), mensajes frozen, rebotes, detección de posible abuso/spam saliente.
- `checks/dns.sh` : resolución de zonas gestionadas por bind, comprobación de registros SOA/NS, coherencia serial.
- `checks/ssl.sh` : días hasta caducidad de cada certificado (Let's Encrypt y manuales), avisos escalonados (30/14/7/1 días).
- `checks/dovecot.sh` : disponibilidad IMAP/POP, autenticación de prueba.
- Entregable: checks de mail/DNS/SSL con avisos escalonados.

Fase 8 — Checks de backups, seguridad e integridad

- `checks/backups.sh` : verifica que `v-backup-user` corrió, antigüedad del último backup por usuario, tamaño anómalo, espacio en destino remoto.
- `checks/security.sh` : lecturas de fail2ban, picos de logins SSH fallidos, cambios en ficheros críticos (hashes de `/etc/passwd` , `sshd_config` , `hestia.conf`) y detección de rootkit opcional.
- `checks/updates.sh` : paquetes de seguridad pendientes y versión de HestiaCP.
- Entregable: checks de backups y seguridad con baseline de integridad.

Fase 9 — Motor de alertas: anti-duplicado, ventanas y kill switch (CRÍTICO)

- **Kill switch hardcoded** `WD_KILL_SWITCH=true` por defecto: cortocircuita todos los notificadores hasta que el usuario lo ponga a `false`. Documentar.
- **Ventana de re-notificación** configurable (p. ej. no repetir la misma alerta antes de N minutos); por defecto conservadora.
- **Estado anti-duplicado fiable:** `SELECT` + `INSERT` con try/catch o guard en memoria por ejecución; nunca confiar solo en `INSERT IGNORE`.
- **Agrupación (digest):** colapsar múltiples alertas de un mismo ciclo en un solo mensaje para evitar avalancha de notificaciones.
- **Escalado:** WARNING → CRITICAL si persiste X ciclos; resolución automática (mensaje "RESUELTO") cuando el check vuelve a OK.
- **Rate limit global:** máximo de notificaciones por hora; al superarlo, se agrupa y se avisa una sola vez.
- Entregable: motor de alertas con estado persistente y pruebas de avalancha.

Fase 10 — Notificadores multicanal

- `notifiers/email.sh`: usa el MTA local / `v-` o `sendmail`; plantilla HTML.
- `notifiers/telegram.sh`: bot token + chat id, vía `curl`.
- `notifiers/webhook.sh`: POST JSON genérico (para integrar con n8n, etc.).
- `notifiers/slack.sh` y `notifiers/discord.sh`: incoming webhooks.
- Interfaz común: todos reciben (`severidad`, `título`, `cuerpo`, `metadatos`).
- Reintentos con backoff y registro de fallos de envío.
- Entregable: al menos email + Telegram + webhook funcionando y testeados.

Fase 11 — Orquestador y planificación (cron)

- `bin/v-watchdog-run`: ejecuta todos los checks activos, agrega resultados, aplica el motor de reglas y dispara notificadores.
- Soporte de ejecución selectiva: `v-watchdog-run --only services,ssl`.
- Bloqueo de concurrencia (flock) para evitar solapamiento de ejecuciones.
- Registro de tarea vía `v-add-cron-job` (intervalo configurable, p. ej. 5 min para servicios, diario para SSL/backups).
- Modo `--dry-run` que evalúa pero no notifica (clave para pruebas seguras).
- Entregable: orquestador con cron instalado y `--dry-run`.

Fase 12 — Comandos CLI del plugin (estilo HestiaCP)

- `v-watchdog-run` — ejecución completa o selectiva.

- `v-watchdog-status` — estado actual de todos los checks (tabla/JSON).
- `v-watchdog-list-alerts` — alertas activas e histórico reciente.
- `v-watchdog-ack <id>` — silenciar/reconocer una alerta.
- `v-watchdog-test-notify <canal>` — enviar notificación de prueba.
- `v-watchdog-config-get/set` — gestionar configuración.
- Salida con soporte `json|shell|plain` como los comandos nativos.
- Entregable: set de comandos CLI documentado con `--help`.

Fase 13 — Interfaz web integrada en el panel

- Página en `web/` con dashboard: semáforo por categoría, métricas y gráficas ligeras (CPU/RAM/disco/correo) usando datos del estado persistido.
- Listado de alertas con filtros por severidad, host y fecha; botón "ACK".
- Formulario de configuración (umbrales, canales, kill switch) que escribe en `watchdog.conf` mediante los comandos CLI (no editar ficheros desde PHP).
- Integración con la navegación de HestiaCP (menú/permiso solo para `admin`).
- Escape de salida obligatorio en plantillas; tokens CSRF en formularios.
- Entregable: UI funcional y protegida.

Fase 14 — Sistema de logging, depuración y rotación

- Logs propios en `plugins/watchdog/logs/` con niveles (DEBUG/INFO/WARN/ERROR).
- Modo debug activable desde configuración: traza completa de cada check y de cada intento de notificación (sin exponer secretos/tokens).
- Rotación con `logrotate` o rotación interna por tamaño/edad.
- Comando `v-watchdog-debug on|off`.
- Entregable: logging robusto y rotado, sin secretos en claro.

Fase 15 — Pruebas, simulación de fallos y validación

- Tests unitarios de la lib (bats) y de cada check con fixtures.
- **Inyección de fallos controlada:** parar un servicio de prueba, llenar disco en un loop device, vencer un cert de staging, encolar correo de prueba — verificar que el watchdog detecta, alerta una sola vez y resuelve.
- **Test de avalancha:** simular 100 dominios caídos y comprobar que el motor agrupa y respeta el rate limit (no envía 100 mensajes).
- Pruebas de idempotencia y de concurrencia (dos ejecuciones simultáneas).
- Entregable: suite de pruebas y reporte de cobertura de escenarios.

Fase 16 — Auto-recuperación (self-healing) opcional y segura

- Acciones de remediación opt-in y conservadoras: reiniciar un servicio caído vía `v-restart-service` con límite de reintentos y backoff.
- Lista blanca de servicios auto-recuperables; nunca acciones destructivas.
- Registro y notificación de toda acción automática ejecutada.
- "Circuit breaker": si un servicio se cae N veces en M minutos, dejar de reintentar y escalar a humano.
- Entregable: self-healing acotado, auditable y desactivable.

Fase 17 — Instalación, desinstalación y compatibilidad

- `install.sh` : comprueba versión de HestiaCP y dependencias, copia ficheros, registra comandos en el PATH del panel, instala cron y crea estado inicial.
- `uninstall.sh` : elimina cron, ficheros y opcionalmente estado/logs, sin romper el panel.
- Compatibilidad con Debian/Ubuntu soportados por HestiaCP y con apache/nginx.
- Idempotencia del instalador (re-ejecutable sin duplicar cron ni estado).
- Entregable: instalador/desinstalador probados en VM limpia.

Fase 18 — Documentación, hardening final y entrega

- Generar **3 documentos** `.md` : Técnico, Profesional de desarrollo y Guía de usuario (instalación, configuración, canales, kill switch, troubleshooting).
 - Revisión de seguridad: permisos de ficheros (640/750), secretos fuera del repo, validación de entradas, sin ejecución de comandos con datos sin sanitizar.
 - Checklist final: kill switch documentado, ventanas anti-avalancha activas, `--dry-run` verificado, rollback claro.
 - Entregable: paquete listo para producción + documentación completa.
-

Anexo A — Matriz de checks (resumen)

Check	Frecuencia	Severidad máx	Auto-heal	Notifica
Servicios	5 min	CRITICAL	Sí (opt)	Sí
Recursos	5 min	WARNING	No	Sí
HTTP dominios	10 min	CRITICAL	No	Sí
Bases datos	10 min	CRITICAL	No	Sí
Cola Exim	15 min	WARNING	No	Sí
SSL caducidad	Diario	WARNING	No (renew opc)	Sí
DNS	30 min	WARNING	No	Sí
Backups	Diario	CRITICAL	No	Sí
Seguridad	15 min	CRITICAL	No	Sí

Anexo B — Riesgos clave

- **Avalancha de notificaciones** → mitigado por motor de alertas (Fase 9).
- **Falsos positivos por flapping** → histéresis y N lecturas consecutivas.
- **Acciones destructivas de self-heal** → lista blanca + circuit breaker.
- **Carga del propio watchdog** → ejecución escalonada y timeouts en cada check.